UNIFAAT 2026.2

03 - Vite, Forms e Listeners

Prof. Luan Lourenço

O repositório do bimestre

`github.com/luan-tavares/unifaat-frontend-bimestre-01`

A partir desta aula o projeto ganha a tabela de Usuários e um CRUD completo — com a API ainda aberta.

O que é um build?

O processo que transforma o código-fonte em arquivos prontos para produção.

```
# 1 · código-fonte
múltiplos arquivos, sintaxes modernas,
TypeScript
```

```
# 2 · build tool
empacota, otimiza e transforma
o código
```

```
# 3 · produção
JS puro, compatível com o navegador
```

```
# exemplos de build tools
Vite · Webpack · Parcel · esbuild
```

Vite: para que serve?

```
# hot module reload
atualiza só o módulo que mudou —
sem recarregar a página
```

```
# pré-compilação
escreva TypeScript e sintaxes modernas;
o Vite transforma em JS puro
```

A pré-compilação é o recurso mais relevante pro curso. O HMR resolve a produtividade do dia a dia.

HMR: sem recarregar a página

```
# sem hmr
salvar → recarregar a página inteira
→ perder estado (formulário, scroll)
```

```
# com hmr
o Vite detecta o módulo alterado
→ envia só ele ao navegador
→ estado preservado
```

vite.config.js

```
export default defineConfig({
  root: 'public',
  server: {
    hmr: true,
    host: true,   // aceita conexões de fora do localhost
    port: 5173
  }
})
```

Executando o Vite

Instalado como dependência **local**: o binário fica em `node_modules/.bin`.

terminal

```
# caminho relativo — funciona sempre, sem depender do PATH
$ ./node_modules/.bin/vite
# via npm script — o npm adiciona .bin ao PATH sozinho
$ npm run dev
```

Eventos: revisão e catálogo

console.js

```
botao.addEventListener("click", function (event) {
  console.log(event.target);  // onde ocorreu
});
// target → onde ocorreu · currentTarget → onde o listener está
```

```
# mouse
click · dblclick
mouseover · mouseenter
```

```
# teclado, input, formulário e página
keydown · keyup · input · change
submit · focus · blur · DOMContentLoaded
```

`preventDefault` e `stopPropagation`

preventDefault.js

```
link.addEventListener("click",
  function (event) {
    event.preventDefault();
// o link não navega
  }
);
```

stopPropagation.js

```
item.addEventListener("click",
  function (event) {
    event.stopPropagation();
// o evento para aqui
  }
);
```

`preventDefault()` cancela o comportamento padrão e o evento **continua** propagando. `stopPropagation()` interrompe o bubbling: listeners dos pais não disparam.

Fases do evento

dom-tree

```
① capturing ↓   window → html → body → main → section → div
② target       button  ← alvo do evento
③ bubbling  ↑   div → section → main → body → html → window
```

Por padrão, `addEventListener` escuta na fase de **bubbling**. Pra capturing: `{ capture: true }` como terceiro argumento.

É por isso que o clique no filho também dispara os listeners dos pais.

Formulários HTML

index.html

```
<form id="form-usuario">
  <input type="text" name="nome" placeholder="Nome" required>
  <input type="email" name="email">
  <input type="checkbox" name="ativo" checked>
  <input type="text" name="id" value="123" readonly>
  <button type="submit" disabled>Salvar</button>
</form>
```

`value` valor atual · `placeholder` exemplo com campo vazio · `disabled` não é enviado · `readonly` não edita, mas é enviado · `required` / `name` validação e identificação.

O evento submit

console.js

```
const form = document.querySelector("#form-usuario");
form.addEventListener("submit", function (event) {
// sem isso, a página recarrega e os dados se perdem
  event.preventDefault();
  const nome = form.nome.value;
  const email = form.email.value;
  console.log(nome, email);
});
```

Listener no `<form>` — não no botão, não nos inputs.

JSON: a mesma lógica desde a aula 01

```
# rede
os dados trafegam como string
```

```
# parsing
string JSON → objeto JS
```

Depois do parsing, vira objeto manipulável — no frontend ou no backend.

O Axios faz o parsing automaticamente: `response.data` já é o objeto.

Axios: cliente HTTP com Promises

terminal

```
$ npm install axios
```

usuarios.js

```
import axios from "axios";
// GET: lista usuários — response.data já vem parseado
const response = await axios.get("http://localhost:3000/usuarios");
// POST: cria um usuário
await axios.post("http://localhost:3000/usuarios", { nome: "Ana", email: "ana@email.com" });
```

Baseado em Promises (`async`/`await`) e com a mesma API no frontend e no Node.

CRUD de usuários

A API está aberta — a autenticação entra na próxima aula.

```
# GET
listar usuários
```

```
# POST
criar usuário
```

```
# PUT
atualizar usuário
```

```
# DELETE
remover usuário
```

Vite, listeners, forms e Axios.

Fechamos o ciclo de comunicação entre frontend e backend. A seguir: **navegadores web e API** — o objeto `window`, cookies, JWT e CORS.

tarefa

TF 03: Vite, Forms e Listeners

Entrega pelo formulário abaixo —
docs.google.com/forms/d/e/1FAIpQLSfVSiBxATYNmBNf0aLWIwGsq7BChyBwGIvV4y6BqKFiTUySAg/viewform